ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

LLM Benchmark project Report

Professors:

Michele Lombardi

Allegra De Filippo

Students:

Simone Migliaccio (matr.: 0001159303)

Simone Spezia (matr.: 0001167605)

Alberto Varini (matr.: 0001189363)

Contents

1 Introduction

The main goal of this project is to provide a general benchmark to evaluate different LLMs performances under different types of planning domains to better understand in advance what are the planning capabilities of a given LLM. In the following sections the main features of this projects are going to be shown.
Also a quick PDDL starting guide has been created to better gain the necessary knowledge about PDDL this specific language used to define planning problems.

2 General Structure

The root folder contains various sub folders, specifically:

- analysis
- archive
- Benchmark Framework
- results

2.1 Benchmark Framework structure

The folder Benchmark Framework is the core of all the work we've done. The main goal of this framework was to provide a easy to use and general testing platform to be employed when evaluating the performances of a LLM is needed, The following pages are going to be used to show the folders and the structure of the framework:

```
LLM Benchmark/
├─ docs/
├─ evaluators/
├─ Leonardo script/
├─ models/
├─ outputs/
├─ prompts/
├─ protocols/
├─ reporting/
├─ runner/
├─ scripts/
├─ slurm logs/
├─ tasks/
├─ tests/
├─ utils/
├─ advanced planning evaluation.py
├─ clear outputs.py
├─ run benchmark.py
└─ setup.md
```

docs Contains a single file for the model preparation which i useful because the benchmark can run on a local PC or on an HPC system such as Leonardo. The important operational rule is that GPU benchmark jobs should not download large model weights. Prepare Hugging Face models first, then run benchmarks from local files.

evaluators Provides tools for to understand, validate and measure the capabilities of LLMs to solve planning problems

Leonardo script Contains specific scripts to launch and manage jobs on Leonardo HPC.

models Manages the communication with various LLMs, using "adapters" which creates a common interface for different backends like NVIDIA API.

outputs folder containing all the output from the LLMs, divided in 3 sub folders *raw, parsed, scored* to better separate respectively the raw output from the model,PDDL extracted and strctured plans and validation and metrics results.

prompts contains text files each one used to explain to the LLM the rules of the domain and how to manage the problem in order to solve it.

protocols defines different strategies for test execution, for example direct plan (which is one shot) or iteratoive repair.

reporting contains scripts for creation of teh plots used in the reports.

runner contains scripts useful to launch a singlke test or the whole suite.

scripts contains scripts for the computation of the numerical score assigned to each domain.

slurm logs Used to gather information for debug when the benchmark runs on Leonardo HPC

tasks PDDL problems and related instances selected for the runs, the instances are divided in 3 categories *easy, medium, hard*.

tests Used to run tests in preparation for the real run of the benchmark.

utils contains executable for the validator to be used during the run of benchmark framework

advanced planning evaluation.py it reads benchmark artifacts from the output folder and writes a model-centric JSON report in the folder results.

clear outputs is used to empty a folder during Leonardo tests

run benchmark is the file that runs the whole sequence of tests

3 PDDL Problems Classification

The first thing to do since we wanted a trustable benchmark and since a lot of PDDL models were already provided as reference material was to classify the problems using a numerical difficulty score. We used a numerical score because it is intuitively understandable by anyone who will ever want to compare the performance of LLMs using these models. Another classification implemented consists into separating problems in different categories based on the type of reasoning they request. To compute the scores the code can be found in the file `score_domains_complexity.py`

3.1 Difficulty Classification

3.1.1 Instances scores

The complexity of a single problem instance is calculated using a weighted logarithmic combination of four key metrics. The use of the logarithm prevents any single feature from dominating the score excessively.

The **Raw Score** is defined as:

$$Raw\_score = 0.45 \cdot \log(1 + A) + 0.20 \cdot \log(1 + G) + 0.20 \cdot \log(1 + N) + 0.15 \cdot T \tag{1}$$

Where:

- **A (Actions):** Number of feasible grounded actions. This is estimated by unifying action parameters with objects that satisfy static preconditions in the initial state.
- **G (Goals):** Total number of atomic conditions in the goal state.
- **N (Numeric Score):** A composite value representing numeric complexity:

  $$N = E_{num} + 2 \cdot P_{num} + 3 \cdot G_{num} + M \tag{2}$$

  where E_{num} are numeric effects, P_{num} numeric preconditions, G_{num} numeric goal conditions, and M is a binary flag for the presence of a metric.
- **T (Topology Score):** Evaluates the spatial/relational complexity based on predicates like *adj* or *connected*:

  $$T = \log(1 + Nodes) + \log(1 + Edges) + Density \tag{3}$$

Obviously a different weight could be assigned to each different part of the computation of raw scores depending on the operator that is doing the tests in order to higlight more for example the number of action present in the instances and so on.
Finally, an **Instance Score (0-100)** is calculated by normalizing the raw score relative to the minimum and maximum scores found within the same domain. The results can be found in the folder `domains_complexity`.
Note that also some additional statistics for each domain are computed based on the results obtained from the instances, for example it could be of interest evaluating the mean and the median of the scores obtain form the instances of a single domain.

3.1.2 Domain scores

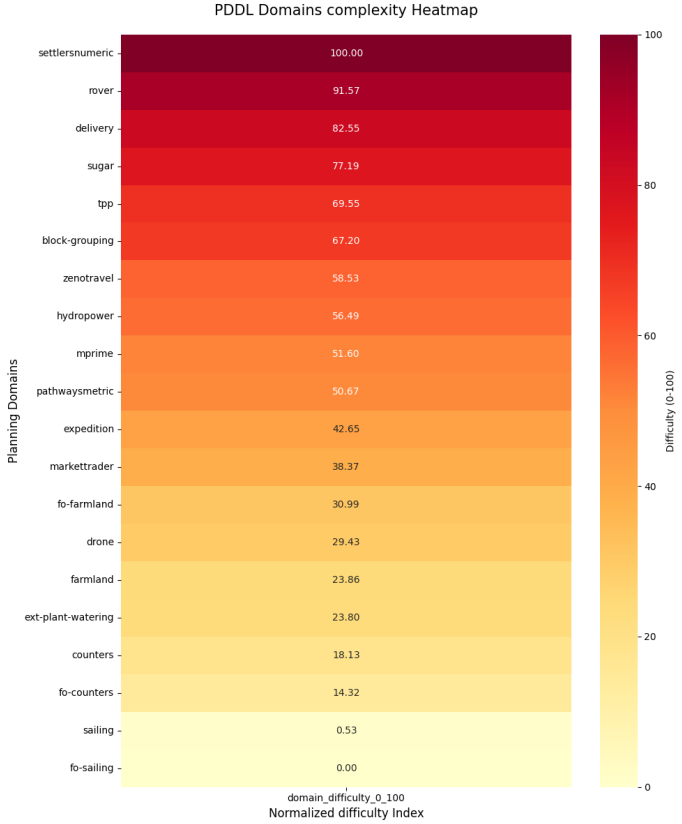
The overall difficulty of a domain is not just the average of its instances, but a reflection of its total state-space and constraint complexity. The **Domain Difficulty (0-100)** is computed by:

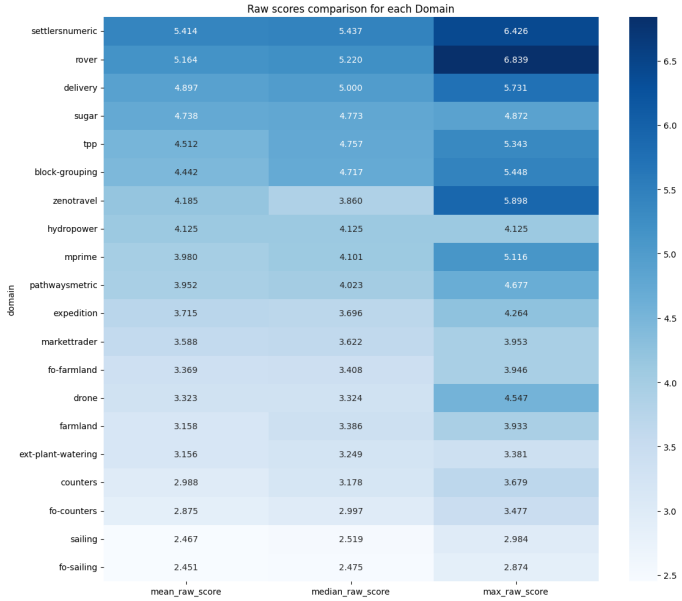
1. Summing the S_{raw} of all instances belonging to the domain to obtain a *Total Raw Score*.
2. Normalizing this total across all tested domains in the benchmark.

This approach ensures that a domain with many complex instances is ranked higher than one with few simple instances, note that obviously the easiest problems gets a domain difficulty equal to 0 (in this case is the "fo-sailing" problem) and the problem with the highest difficulty has domain difficulty equal to 100, (in our case the "settlersnumeric" problem).

3.1.3 Results obtained

In order to obtain a visual representation of the classified problems, we produced heatmaps for both the overall normalized domain scores and the raw scores.





3.2 Reasoning Classification

Here the classification based on the type of reasoning these problems require the LLM to employ is shown. We decided to split the reasoning types into 5 categories:

- **Spatial:** These problems are typically about objects that have to be moved in a 2D (or more dimensional) spaceframe.
- **Numerical Optimization:** These problems require from the LLM an extensive capacity

to deal with numerical optimization problems, such as managing complex cost functions and resource constraints.

- **Temporal Planning:** In this kind of problem it is required from the LLM to be able to reach the goal defining an order between objects having timing constraints and relations.
- **Logistic:** In this kind of problems it is required to reach the goal by planning some kind of goods transportation and agents coordination.
- **Logic:** These are abstract puzzles where the state space and actions represent symbolic or mathematical transformations rather than physical interactions. They test the model's ability to reason over purely formal rules without relying on world knowledge.

In the following table the classification of the 20 planning problems is shown, note that in this case all the work have been done by hand and that the problems in the table are ordered form hardest to easiest following the previous difficulty classification. Note also that the evaluation is based on how much the problem belong to the specific domain, for example in the problem "settlersnumeric" there is a dominant Numerical Optimization part hence the problem is going to be classified as a Numerical Optimization problem.

PDDL Problem	**Reasoning Category**
Settlersnumeric	Numerical Optimization
Rover	Logistic
Delivery	Logistic
Sugar	Numerical Optimization
Tpp	Numerical Optimization
Block-Grouping	Spatial
Zenotravel	Logistic
Hydropower	Temporal Planning
Mprime	Logic
Pathwaysmetric	Numerical Optimization
Expedition	Logistic
Markettrader	Numerical Optimization
Fo-farmland	Numerical Optimization
Drone	Spatial
Farmland	Numerical Optimization
Ext-Plant-Watering	Spatial
Counters	Numerical Optimization
Fo-Counters	Numerical Optimization
Sailing	Spatial
Fo-Sailing	Spatial

Table 1: Reasoning Category Classification Tab

3.3 Problems chosen

In order to chose the correct problem to be solved by LLMs we decided to pick 3 sets composed by 3 problems each. The sets are composed by 3 problems with different reasoning categories and specifically from the categories Numericaal Optimization, Spatial and Logistic. Problems related to Temporal Planning and Logic were not chosen since just one problem for each type was present; we suggest, in order to study the capabilities of LLMs in these domains, to use a PDDL problenms dataset specifically containing these kind of problems.
In order to better understand how we picked the problems the following tables are shown, the elements are ordered with decreasing difficulty.

Numerical Optimization
Settlersnumeric
Sugar
Tpp
Pathwaysmetric
Markettrader
Fo-farmland
Farmland
Counters
Fo-Counters

Spatial
Block-Grouping
Drone
Ext-Plant-Watering
Sailing
Fo-Sailing

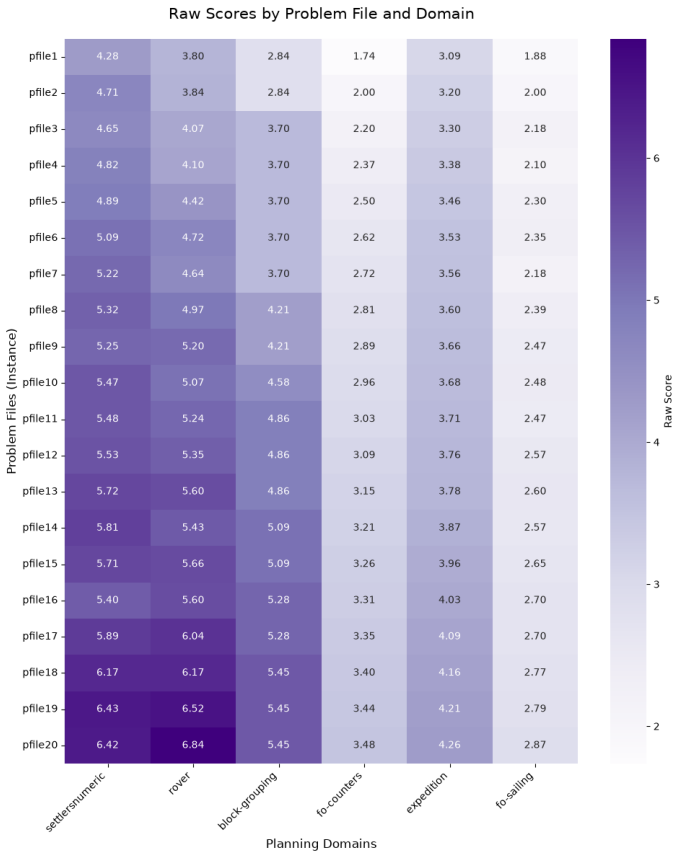
Logistic
Rover
Delivery
Zenotravel
Expedition

Table 2: Reasoning Categories Selected for Benchmark

In order to obtain a more clear output from the LLM we decided to select two differents sets of three problems each and specifically:

- **Hard Set:** composed by *settlersnumeric, block-grouping, rover*
- **Easy Set:** composed by *fo counters, fo sailing, expedition*

It is clearly visible that we picked respectively the most three difficult problems and the three easiest ones. This has been done to ensure also in the outputs a visible difference in the metrics when evalauting the outputs.
Additionally it can be seen from the following heatmap that the instances of these domains have increasing difficulty:



It is clearly visible both the difference in terms of difficulty between the Hard set and the Easy set and the increasing difficulty related to the pfiles; because of this last consideration we applied an additional, partition to divide the pfiles in three different categories and specifically:

- Easy (composed by pfiles from 1 to 4)
- Medium (composed by pfiles from 8 to 11)
- Hard (composed by pfiles from 15 to 18)

Also it is worth noticing that we did not use all the instances available for each selected domain but we decided to choose a small batch (4 instances) of instances representing the easy, medium and hard categories.

4 Configured LLMs

The benchmark framework was designed to support two execution backends: Leonardo HPC by CINECA and the NVIDIA API. Leonardo was used for local execution of Hugging Face models, while the NVIDIA API provided remote access to additional models without requiring every checkpoint to be installed and hosted locally.

4.1 Execution Backends

4.1.1 Leonardo HPC

Leonardo HPC was the main platform considered for local model execution. The models were loaded through the Hugging Face backend, allowing direct control over the execution environment, GPU resources, precision, prompt formatting, and generation parameters.

The selection of locally executable models was supported by whichLLM, a repository used to estimate which LLMs can run on a given hardware configuration and to compare models by release recency. This was useful to identify models that were realistic candidates for the available Leonardo resources before running the benchmark.

4.1.2 NVIDIA API

The NVIDIA API was used as a remote inference backend. This reduced the setup effort required to test recent open-weight models and made it possible to include models that would have been less convenient to install or run locally.

The benchmark pipeline remained the same across both backends: prompts, parsing, validation, and scoring followed the same workflow, while only the model adapter changed. The NVIDIA API backend also introduced practical constraints, such as rate limits, temporary service availability, and model-specific API key requirements.

4.2 Selection Criteria

The models were selected according to three practical criteria:

- **Hardware feasibility:** local models had to be compatible with the available Leonardo environment, using whichLLM as a practical reference for hardware requirements.
- **Recency:** priority was given to recent LLM families, using whichLLM as a reference to compare model release dates.
- **Model diversity:** the benchmark included models from different families to avoid evaluating a single architectural lineage.

Using both backends made the framework more flexible than a Leonardo-only setup. Leonardo provided controlled local execution, while the NVIDIA API allowed the benchmark to cover additional hosted models through the same interface.

4.3 Configured Models

The implemented model set includes the models configured for experimentation in the framework. Since the benchmark runs were still being refined, this section describes the configured model pool rather than treating any single run as the definitive final result.

Backend	Execution mode	Configured models
Leonardo HPC	Local HF	Gemma 4 31B, Phi-4, Nemotron Nano, Qwen 3.6
NVIDIA API	Remote hosted	Phi-4 Mini, GPT-OSS 120B, Kimi K2.6, Llama 3.3 70B, Nemotron Nano, Qwen 3.5

Table 3: LLM models configured in the benchmark framework.

5 Benchmark output Parsing and Output Structure

DA RIEMPIRE

6 Output Validation Using VAL

DA RIEMPIRE

7 LLMs Reasoning Capabilities Evaluation

7.1 Pipeline and Data flow

How Raw / Parsed / Scored become one DataFrame , which Classes hold the state the metrics need and why the load → row → aggregate → plot → report stages. This section maps the analysis script `advanced_planning_evaluation_sp.py` onto the rest of the Repo , showing how the three on-disk artifact layers Raw / Parsed / Scored. become DataFrames , which data classes exist and why and how the data-manipulation stage hands off to metric computation and visualization.

7.1.1 Pipeline Generic description

Everything funnels into one DataFrame `df_metrics` , which is the single source of truth for every groupby and every plot. we start from the outputs:

- raw : generation text + reasoning text
- parsed : raw.actions + reasonig.actions
- scored : VAL outputs solved , prefix lenght , valid@k informations
- tasks : domain.pddl + pfileN.pddl

the `look_artifact_rows()` takes the data present in Parsed and then joins Raw and Scored data in a single row for each instance , collapsing the retry loop by scanning the attemps back to front keeping the last non empty plan as the representative `_actions` , while preserving the full attemp list `_parsed_attemps` , `_scored_attemps` , `_raw_attemps` as private colums so per-iteration metrics can still be computed downstream. Keeping the grain at instance level leads to every aggregate metric (SR , HR , PS) bein an average over comparable units , while maintaining the iteration structure in the private `_*_attemps` and used as option by the metrics that explicitly need it like (FASR , IWSR , iteration profile , CoTal). For each row is also added Validity and Iteration count taken from the Scored file `Valid = scored["solved"] , Iterations = scored["iterations_used"]` , the presence of the CoT is instead taken from `\protocols\protocol.yaml` or instead as a fallback is looked the presence of reasonig actions or text since we want to compute the Cot alignement metric only for those models that have it available. the function `look_artifact_rows()` for each row the `df_raw` informations:

- Model
- Domain
- Problem
- Difficulty
- Protocol
- Run_id
- Valid
- Length
- Iterations
- Chain_of_Thought
- actions
- _cot_text
- _parsed_attempts
- _scored_attempts
- _raw_attempts

- _file_path
- parsed_file_path

7.2 Metrics scientific foundation

7.3 Metric Families

7.3.1 Planning Efficiency

7.3.2 Execution and Reasoning

7.3.3 Grounding and Discriminatory

7.3.4 PS (Composite Planning Score)

7.4 Cross-Domain and Cross-LLM comparison

8 Conclusions

DA RIEMPIRE